Blink-Anbindung

Lernunterlage: Geheimnisse sicher übergeben, Token-Rotation und ein eigener Client für eine inoffizielle API

1 Worum es geht

Die Übersicht zeigte „Kameras (Blink): Nicht eingerichtet“. Ursache: Der Erst-Login war auf diesem Rechner nie gelaufen. Auf dem Weg dorthin kamen drei Themen dazu: Geheimnisse sicher übergeben (REVIEW.md **M-4**), ein Login, der an Kleinigkeiten scheiterte, und eine Bibliothek, die Blink nicht mehr akzeptiert (**M-5**).

2 Geheimnisse nie als Argument (M-4)

Der Spike hat das Blink-Token früher so gespeichert:

```
dotnet user-secrets set Blink:InitialRefreshToken <TOKEN> …
                                                  ▲ Token steht in der Prozessliste
```

Jedes Programm auf dem Rechner kann die Argumente laufender Prozesse lesen (`ps aux`, `/proc/*/cmdline`). Jetzt geht ein JSON-Objekt über **stdin**, das sieht nur der Empfänger:

```
using var process = StartUserSecrets("set", secretsId, redirectInput: true);
await process.StandardInput.WriteAsync(JsonSerializer.Serialize(values));
process.StandardInput.Close();     // Ende der Eingabe
```

Weg	Wer kann mitlesen?	Hier
Kommandozeilenargument	alle Prozesse (Prozessliste), Shell-History	nie für Geheimnisse
stdin (Pipe)	nur der empfangende Prozess	Spike, *-einrichten.sh
verdeckte Eingabe	niemand (erscheint als *)	Blink-Passwort
Datei mit Rechten 600	nur der eigene Benutzer	User Secrets

Außerdem behält der Spike **kein** eigenes Token mehr: Nach dem ersten Login des Backends tauscht Blink das Token aus, eine Kopie wäre wertlos und trotzdem ein Geheimnis im Klartext.

3 Fehler, die erst im echten Betrieb auffielen

Problem	Ursache	Lösung
Secrets landeten im falschen Projekt	relative Pfade, stimmten nur aus tools/BlinkSpike	--id <UserSecretsId> statt --project
Jeder Versuch ein neues Gerät	HardwareId erst nach Erfolg gespeichert	beim ersten Start sofort speichern
E-Mail in der Shell-History	Anleitung: user-secrets set … E-Mail	Spike fragt selbst, speichert über stdin
Unklar, welcher Code läuft	keine Versionsangabe	Versionszeile beim Start

Fehlversuche sind bei fremden Diensten teuer: Blink sperrt nach 5 Fehlversuchen für 10 Minuten. Deshalb erst prüfen (--email-pruefen, Login in der Blink-App), dann einmal versuchen.

4 Eine Bibliothek, die Blink nicht mehr akzeptiert (M-5)

Blink.NET 1.0.9 (die neueste Version) bekam für die Kameraliste `426 Upgrade Required`. Im dekompilierten Code war zu sehen: alter Endpunkt `/network/{id}/cameras` und ein alter Android-App-Build im Header. Das aktive Projekt **blinkpy** liest stattdessen den **Homescreen** und schickt keinen App-Build. Optionen:

Option	Bewertung
(a) neuere Version	gibt es nicht
(b) Header per Reflection ändern	zerbrechlich, nicht erklärbar, löst evtl. nichts
(c) eigener schlanker Client (gewählt)	volle Kontrolle, testbar, weniger fremder Code im Dauerdienst

```
1. POST api.oauth.blink.com/oauth/token     Refresh-Token → Access Token (+ neues Refresh-Token)
2. GET  rest-prod…/users/tier_info          → Server (Tier) und Account-Id
3. GET  rest-{tier}…/accounts/{id}/homescreen → alle Kameras in EINER Antwort
```

5 Token-Rotation und der Schutz vor Schleifen

- **Rotation:** Bei jeder Erneuerung gibt Blink ein neues Refresh-Token, das alte ist dann ungültig. Wird das neue nicht gespeichert, ist nach einem Neustart ein Login mit 2FA nötig. Deshalb wird es **sofort** verschlüsselt in die Datenbank geschrieben (Data Protection), bevor irgendetwas anderes passiert.
- **Nur eine Erneuerung gleichzeitig** (`SemaphoreSlim`): Zwei parallele Erneuerungen mit demselben Refresh-Token würden sich gegenseitig ungültig machen.
- **401 beim Abruf:** einmal erneuern und wiederholen. Ein zweites 401 ist ein Fehler, keine Schleife.
- **Abgelehntes Refresh-Token:** wird nie wieder benutzt. Erst ein neues Token (BlinkSpike) löst einen neuen Versuch aus, ohne Neustart.
- **426 oder Netzwerkfehler** sind kein Login-Problem: Das Token bleibt gültig.
- **Der Tier** landet im Hostnamen. Er wird geprüft (nur a–z, 0–9, Bindestrich), damit eine seltsame Antwort uns nie an einen fremden Server schickt.

Token, Bild-Adressen und Seriennummern kommen nie ins Log. Beim ersten Abruf stehen dort nur die NAMEN der Felder, damit sich die Zuordnung an echten Daten prüfen lässt, ohne zu raten.

6 Tests

12 Tests mit `FakeBlinkHandler` (spielt die Blink-Cloud, ohne Internet): Kameras aus dem Homescreen, kein App-Build-Header, rotiertes Token verschlüsselt in der DB und nicht im Log, nur eine Erneuerung bei fünf parallelen Aufrufen, 2-Minuten-Cache, einmal erneuern bei 401, kein Kreisel bei zweimal 401, neuer Startwert ohne Neustart, 426 ist kein Login-Problem, kein fremder Server, ohne HardwareId nicht eingerichtet.

Danach einmal gegen das echte Konto: Status **connected**, zwei Kameras, Token verschlüsselt in der Datenbank.

7 Kurzantworten für Rückfragen

Frage	Antwort
Warum stdin statt Argument?	Argumente stehen in der Prozessliste, stdin sieht nur der Empfänger.
Warum ein eigener Client?	Die Bibliothek wird von Blink abgelehnt, eine neuere gibt es nicht.
Was ist Token-Rotation?	Jede Erneuerung liefert ein neues Refresh-Token, das alte wird ungültig.
Warum keine Schleife bei 401?	Wiederholte Fehlversuche führen bei Blink zur Sperre.
Was ist das Risiko?	Inoffizielle API: Blink kann sie jederzeit ändern. Dann muss der Client angepasst werden.

8 Merksätze

- Geheimnisse nie in Argumente, Shell-History oder Logs.
- Rotierte Tokens sofort speichern, nur eine Erneuerung gleichzeitig.
- Bei 401 einmal erneuern, dann aufhören.
- Fremden Code ansehen, bevor man ihm Zugangsdaten gibt (hier: dekompiliert).
- Bei inoffiziellen APIs nichts raten: Feldnamen an echten Daten prüfen.